

AuraGreet Data Journey

A comprehensive end-to-end technical overview detailing data transit, storage execution layers, API proxies, and local storage state persistence.

1. Complete Traffic Flow Overview

The sequence below traces the logical path of data transactions during name storage operations:

1

BROWSER → LOCALSTORAGE

State Hydration on Load

When the page loads, the frontend reads `saved_guest_name` from LocalStorage. If found, the screen immediately renders "Hello [Name]" instead of "Hello Guest".

2

USER INPUT → INPUT BUFFER

Isolated Keystroke Buffering

As the user types, characters accumulate in the text field buffer. The visual screen greeting remains unchanged until submission to keep the interface polished.

3

USER SUBMIT → NETWORK & LOCAL VERIFICATION

Cloud Query Response & Display

Clicking Submit or hitting Enter intercepts the event. The frontend fires the HTTPS POST request (with a `.select()` payload request). When the database responds with the saved row, the frontend pulls the name from the response, displays it on the screen, and commits it to LocalStorage.

4

NETWORK → KONG API GATEWAY

Gateway Authentication & CORS Inspection

The encrypted HTTPS request lands on Kong. The API gateway verifies access permissions, CORS configurations, and token signatures.

5

KONG GATEWAY → POSTGRES ENGINE

REST API Endpoint to SQL Translation

PostgREST translates the JSON headers and REST routes into standard SQL insert statements, maintaining PostgreSQL database transaction logs.

6

POSTGREST → DATABASE (POSTGRESQL)**Row Level Security (RLS) & Storage Commit**

PostgreSQL runs the query against the `Allow public read and insert` RLS rule. If allowed, it generates a sequential primary ID and writes the row to the Write-Ahead Log (WAL) on disk.

2. Frontend Client Mechanics

The client side acts as the interface, executing capture operations in three main phases:

A. Local State Hydration

The application checks for cached visitor names upon initialization:

```
const savedName = localStorage.getItem('saved_guest_name');
if (savedName) {
  displayName.textContent = savedName;
} else {
  displayName.textContent = "Guest";
}
```

B. Submission Commit

Upon clicking Submit or pressing Enter, the text is sent to the cloud, and the database response is verified before printing to the screen:

```
greetingForm.addEventListener('submit', async (e) => {
  e.preventDefault();
  const cleanedName = userNameInput.value.trim();
  if (cleanedName.length === 0) return;

  // Save to DB and retrieve the actual name returned by the database response
  const savedNameFromDB = await saveName(cleanedName);
  if (savedNameFromDB) {
    displayName.textContent = savedNameFromDB;
    localStorage.setItem('saved_guest_name', savedNameFromDB);
  }
});
```

C. Dynamic Rendering

After logging names, JavaScript executes a query to read the logs. It formats the server datetime stamps, sanitizes names to prevent script injection attacks (XSS), and dynamically inserts table markup elements.

3. Backend Infrastructure (Supabase API Gateway)

Supabase abstracts the backend layer using a dedicated container architecture that acts as a proxy proxying REST transactions to PostgreSQL:

A. Kong API Gateway

Kong is the first layer of contact for the web application. It acts as a gatekeeper, verifying that incoming HTTPS requests contain a valid project API key. It parses request metadata, confirms CORS origins, and enforces rate limit controls.

B. PostgREST Translation

Because relational databases communicate natively via SQL, web browsers cannot query Postgres directly. PostgREST bridges this gap by automatically converting HTTP requests into SQL commands. For example:

HTTP Method	API URL Endpoint	PostgreSQL SQL Translation
POST	/rest/v1/Customer%20Names	INSERT INTO public."Customer Names" ("Name") VALUES ('NameValue');
GET	/rest/v1/Customer%20Names? limit=10	SELECT * FROM public."Customer Names" LIMIT 10;

4. Cloud Database Engine (PostgreSQL)

At the final stage, data enters the PostgreSQL relational database engine which manages execution, integrity, and safety:

A. Row Level Security (RLS) Filter

Before any insertion happens, PostgreSQL inspects permissions. Because RLS is active on the `Customer Names` table, the database checks the query against your security policy:

```
-- The SQL policy validating public access
CREATE POLICY "Allow public read and insert"
ON "Customer Names"
FOR ALL
TO public
USING (true)
WITH CHECK (true);
```

This allows anonymous public connections (authenticated via the JWT key) to safely execute both insert and read operations.

B. Transaction Execution

Once allowed, the database opens a secure transaction. It appends the operation to the **Write-Ahead Log (WAL)** on disk to guarantee transaction safety (Durability). Finally, it writes the record, populates the primary ID sequence, stamps the current server timestamp via `now()`, and commits the transaction.